RAPPORT — INTELLIGENCE ARTIFICIELLE

Utilisation de l'IA dans le projet CV Studio

Outils, prompts et choix techniques

1. Étude de l'existant

Plusieurs solutions existent déjà pour créer un CV :

Solution	Ce qu'elle fait	Limites
Canva, modèles Word	Modèles graphiques à remplir	On écrit tout seul, aucune aide pour le contenu
Europass	CV européen standard en ligne	Formulaire long, style fixe, pas de reformulation
Zety, CVDesignR…	Formulaires et phrases toutes faites	Textes génériques, téléchargement souvent payant
ChatGPT et autres chatbots	Rédige un CV à partir d'un texte	Peut inventer des informations, pas de PDF prêt, pas d'historique

Notre solution : on discute avec un assistant, il pose les questions et améliore les textes ; rien n'est inventé (chaque information est vérifiée) ; le CV sort en PDF et reste dans l'historique.

2. Les outils d'IA utilisés

Outil	À quoi il a servi
Claude (Anthropic), extension VS Code	Écrire le code, tester et corriger les bugs
Gemini (API)	L'assistant qui discute avec l'utilisateur dans le site
Qwen3-0.6B entraîné par nous (LoRA)	Transformer les informations du CV en LaTeX
Ollama	Tester un modèle en local au début du projet

Pourquoi l'extension VS Code plutôt que le chat ? L'extension travaille directement dans le projet : elle lit les fichiers, lance les commandes et les tests, puis corrige. Avec un chat, il faut tout copier-coller et tester soi-même.

3. Où j'ai utilisé l'IA

Étape	Ce que l'IA a fait
Développement	Code du backend (FastAPI), du frontend (Angular), du dataset et de l'entraînement
Tests	Lancer l'évaluation du modèle, tester les routes de l'API, compiler le site
Débogage	Trouver la cause des bugs à partir des journaux et prouver la correction par un test

4. Mauvais prompts et bons prompts

Un bon prompt donne : un **rôle**, le **contexte**, la **tâche**, le **format** de la réponse et les **contraintes**. Pour chaque cas : le mauvais prompt, le bon prompt, puis le résultat.

4.1 Trouver les erreurs dans l'entraînement

Mauvais prompt — PROMPT NON OPTIMISÉ

```
Analyse bien le projet et dis-moi s'il y a une mauvaise méthode.
```

Figure 1 — Mauvais prompt

Problème : pas de rôle, on ne sait pas quoi analyser ni sous quelle forme répondre.

Bon prompt — PROMPT OPTIMISÉ

RÔLE	`Tu es un expert en entraînement de modèles d'IA.`
CONTEXTE	`Mon projet génère des CV. Un petit modèle (Qwen3-0.6B) est entraîné avec train.py et un dataset de 22 exemples. Le CV généré est mauvais.`
TÂCHE	`Lis le code et trouve les erreurs de l'entraînement.`
FORMAT	`Un tableau : erreur, preuve, solution.`
CONTRAINTES	`Ne modifie pas encore le code. Donne une preuve pour chaque erreur.`

Figure 2 — Bon prompt

IA **Réponse de l'IA (extrait)**

- L'entraînement et l'utilisation n'ont pas le même format : le modèle n'apprend pas la bonne tâche.
- 11 exemples sur 22 sont coupés (plus de 512 tokens) : le modèle n'apprend jamais à finir un CV.
- Le modèle apprend aussi à recopier la consigne, pas seulement la réponse.
- Trop peu de données : 22 exemples, aucune donnée de test.

Figure 3 — Réponse : erreurs trouvées avec leurs preuves

Résultat : des erreurs précises et prouvées, qu'on a pu corriger une par une.

4.2 Corriger l'entraînement (fine-tuning)

Mauvais prompt

PROMPT NON OPTIMISÉ

```
Corrige-moi le training, je veux une bonne chose.
```

Figure 4 — Mauvais prompt

Problème : « une bonne chose » ne veut rien dire pour l'IA ; elle ne connaît ni la machine ni le but.

Bon prompt

PROMPT OPTIMISÉ

RÔLE	Tu es un expert en fine-tuning avec Hugging Face.
CONTEXTE	J'ai une carte graphique de 4 Go. Le modèle doit transformer les informations d'un CV en code LaTeX.
TÂCHE	Corrige train.py et crée un dataset : 900 exemples pour apprendre, 60 pour tester.
FORMAT	Le code complet, puis les résultats.
CONTRAINTES	Aucun exemple coupé. Le modèle apprend seulement la réponse. Dis si tu choisis LoRA ou QLoRA et pourquoi.

Figure 5 — Bon prompt

train.py

```python
class CompletionOnlyTrainer(Trainer):
    """Loss uniquement sur la réponse, logits calculés seulement sur cette partie."""

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        # La loss est déjà moyennée par exemple : le Trainer divise par
        # le nombre de pas d'accumulation.
        self.model_accepts_loss_kwargs = False

    def compute_loss(self, model, inputs, return_outputs=False, num_items_in_batch=None):
        input_ids = inputs["input_ids"]
        prompt_len = int(inputs["prompt_len"][0])
        keep = input_ids.shape[1] - prompt_len + 1

        outputs = model(
            input_ids=input_ids,
            attention_mask=inputs["attention_mask"],
            logits_to_keep=keep,
        )
        # Logit à la position t -> prédit le token t+1.
        logits = outputs.logits[:, :-1, :].float()
        targets = input_ids[:, prompt_len:]
        loss = F.cross_entropy(logits.reshape(-1, logits.size(-1)), targets.reshape(-1))
        return (loss, {}) if return_outputs else loss
```

Figure 6 — Code produit : le modèle apprend seulement la réponse

IA

Résultats

- Erreur de validation : 0,087 (avant : 1,83).
- 30 CV de test sur 30 corrects.
- Choix de LoRA et pas QLoRA : le modèle tient déjà dans 4 Go, QLoRA l'aurait abîmé.

Figure 7 — Résultats après correction

Résultat : un entraînement qui marche, avec des chiffres pour le prouver.

4.3 Few-shot : donner des exemples à l'IA

Le few-shot consiste à montrer un ou plusieurs exemples à l'IA avant de lui poser la question.

Exemple : créer le dataset d'entraînement du modèle.

Mauvais prompt (sans exemple) — PROMPT NON OPTIMISÉ

```
Crée-moi un dataset pour entraîner le modèle à écrire des CV.
```

Figure 8 — Mauvais prompt

Problème : sans exemple, l'IA ne sait pas quelle forme donner aux données ni quelle règle suivre. Les textes reformulés risquent d'ajouter des informations qui n'étaient pas dans la description.

Bon prompt (avec un exemple) — PROMPT OPTIMISÉ

RÔLE
```
Tu es un expert en préparation de données pour l'IA.
```

CONTEXTE
```
Le modèle doit apprendre à transformer la description d'un projet en lignes de CV.
```

TÂCHE
```
Écris d'autres projets dans dataset_pools.py, en suivant exactement l'exemple.
```

EXEMPLES
```
{
"name": "Application de gestion de stock",
"technologies": ["Spring Boot", "Angular", "MySQL"],
"raw": ["j'ai développé une appli web de gestion de stock avec spring boot et angular, avec une base mysql. elle gère les produits, les fournisseurs et les alertes de rupture"],
"bullets": ["Développement d'une application web de gestion de stock avec Spring Boot et Angular", "Gestion des produits, des fournisseurs et des alertes de rupture", "Stockage des données dans une base MySQL"]
}
```

FORMAT
```
Du code Python, même structure que l'exemple.
```

CONTRAINTES
```
Les bullets reprennent seulement ce qui est écrit dans raw. Projets variés : web, data, mobile, DevOps.
```

Figure 9 — Bon prompt avec un exemple (one-shot)

dataset_pools.py

```
        {
            "name": "Plateforme de réservation de salles",
            "technologies": ["Spring Boot", "Angular", "PostgreSQL", "JWT"],
            "raw": [
                "plateforme de réservation de salles de réunion. back en spring boot, front
angular, base postgresql, authentification jwt",
                "J'ai réalisé une plateforme pour réserver des salles de réunion. Le back-end
est en Spring Boot, le front en Angular, avec PostgreSQL et une authentification JWT.",
            ],
            "bullets": [
                "Réalisation d'une plateforme de réservation de salles de réunion",
                "Développement du back-end avec Spring Boot et du front-end avec Angular",
                "Mise en place d'une authentification JWT et d'une base PostgreSQL",
            ],
        },
        {
            "name": "Application de gestion de projets freelance",
            "technologies": ["Angular", "Spring Boot"],
            "raw": [
                "J'ai réalisé une application de gestion de projets freelance avec Angular et
Spring Boot.",
                "appli de gestion de projets pour freelances en angular + spring boot",
            ],
            "bullets": [
                "Développement d'une application de gestion de projets freelance avec Angular et
 Spring Boot",
            ],
        },
        {
            "name": "E-banking",
            "technologies": ["Spring Boot", "Angular", "Spring Security", "MySQL"],
            "raw": [
```

Figure 10 — Code obtenu : nouveaux projets écrits par l'IA sur le modèle de l'exemple

Résultat : l'IA a suivi exactement la structure de l'exemple, et chaque ligne de CV reprend seulement ce qui est dans la description. Le dataset final contient 900 exemples pour l'entraînement et 60 pour les tests.

4.4 Débogage : le CV contenait des informations inventées

Mauvais prompt — PROMPT NON OPTIMISÉ

```
Regarde, il ne comprend pas bien, pourquoi ?
```

Figure 11 — Mauvais prompt

Problème : on ne dit pas ce qui est faux ni ce qu'on attend.

Bon prompt — PROMPT OPTIMISÉ

RÔLE	Tu es un expert en débogage Python.
CONTEXTE	Le CV contient des compétences que je n'ai jamais données (Python, React, AWS) et une mauvaise date.
TÂCHE	Trouve la cause, corrige, puis teste.
FORMAT	La cause en une phrase, le code corrigé, le résultat du test.
CONTRAINTES	Ne bloque pas la conversation. Accepte mes fautes de frappe.

Figure 12 — Bon prompt

guards.py

```python
def skill_mentioned(skill, text):
    """Compétence présente dans le texte, fautes de frappe tolérées ('flutyter', 'langraph')."""
    target = norm(skill)
    if not target or target in NOT_SKILLS:
        return False
    if len(target) <= 2:  # "C", "R", "Go" : mot exact (pas le "c" de "c'est")
        return
    bool(re.search(rf"(?<![a-z0-9+#'’]){re.escape(ascii_lower(skill))}(?![a-z0-9+#'’])",
                           ascii_lower(text)))
    if target in norm(text):
        return True
    words = re.findall(r"[a-z0-9+#.]+", ascii_lower(text))
    grams = {"".join(words[i:i + n]).replace(".", "") for n in (1, 2, 3) for i in
    range(len(words) - n + 1)}
    return any(SequenceMatcher(None, target, g).ratio() >= 0.82 for g in grams if abs(len(g) -
    len(target)) <= 3)


# ---------------------------------------------------------------------
```

Figure 13 — Code produit : on garde une compétence seulement si l'utilisateur l'a écrite

IA **Résultat du test**

- Cause : le modèle complète les champs avec des valeurs qui semblent logiques.
- Supprimés : téléphone inventé, compétences jamais citées, chiffre « 40 % ».
- Corrigé : « ETAP » et « D'étap » regroupés en un seul stage.

Figure 14 — Test de la correction

Résultat : plus aucune information inventée dans le CV.

4.5 Comptes utilisateurs et historique (CRUD)

Mauvais prompt — PROMPT NON OPTIMISÉ

```
Ajoute un login et l'historique des CV.
```

Figure 15 — Mauvais prompt

Problème : rien sur la sécurité ni sur les opérations voulues.

Bon prompt — PROMPT OPTIMISÉ

RÔLE	Tu es un développeur web expert (FastAPI et Angular).
CONTEXTE	Mon site de CV n'a pas de comptes utilisateurs.
TÂCHE	Ajoute l'inscription, la connexion et l'historique des CV : ajouter, voir, supprimer.
FORMAT	La liste des routes, le code, puis des tests.
CONTRAINTES	Mot de passe chiffré. Chaque utilisateur voit seulement ses CV. Utilise SQLite.

Figure 16 — Bon prompt

app/api.py

```python
@app.post("/auth/register", response_model=AuthOut, status_code=201, tags=["comptes"])
def register(body: RegisterIn):
    validate_registration(body.name, body.email, body.password)
    if db.get_user_by_email(body.email.strip()):
        raise HTTPException(409, "Un compte existe déjà avec cette adresse e-mail.")
    user = db.create_user(body.name.strip(), body.email.strip(), hash_password(body.password))
    return AuthOut(token=create_token(user), user=_user_out(user))


@app.post("/auth/login", response_model=AuthOut, tags=["comptes"])
def login(body: LoginIn):
    user = db.get_user_by_email(body.email.strip())
    if not user or not verify_password(body.password, user["password_hash"]):
        raise HTTPException(401, "E-mail ou mot de passe incorrect.")
    user = db.get_user(user["id"])  # sans le hash du mot de passe
    return AuthOut(token=create_token(user), user=_user_out(user))
```

Figure 17 — Code produit : inscription et connexion

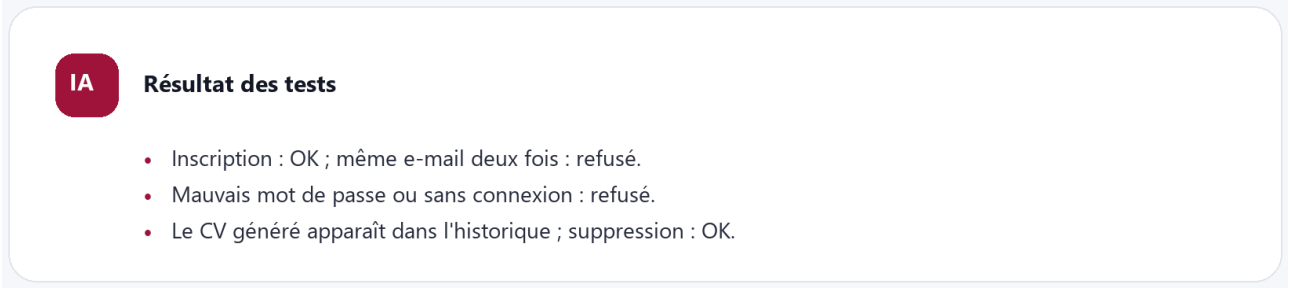


Figure 18 — Tests de la partie CRUD

Résultat : un CRUD sécurisé et testé.

5. Choix des technologies

Technologie	Pourquoi
Python	Le langage de l'IA : il a les bibliothèques pour entraîner des modèles (PyTorch, Transformers)
PyTorch, Transformers, PEFT	Entraîner le modèle avec LoRA facilement
Qwen3-0.6B	Petit modèle gratuit, rapide, qui tient sur une carte graphique de 4 Go
Gemini	Comprend bien le français, répond vite, version gratuite
FastAPI	API simple et rapide, avec une documentation automatique
SQLite	Base de données légère, sans installation
LaTeX	Donne des CV au rendu très professionnel
Angular	Framework complet pour un site organisé en pages et composants